

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Bảo vệ luận văn Thạc sĩ — Ngành Công nghệ Thông tin

Học viên: **Bùi Văn Dũng** — MSHV 220201014

TP. Hồ Chí Minh, 2026

Trường Đại học Công nghệ Thông tin — ĐHQG TP. Hồ Chí Minh

Cán bộ hướng dẫn: **TS. Nguyễn Văn Dũ** · **TS. Đỗ Trí Nhựt**

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Bảo vệ luận văn Thạc sĩ — Ngành Công nghệ Thông tin

Học viên: **Bùi Văn Dũng** — MSHV 220201014

TP. Hồ Chí Minh, 2026

Trường Đại học Công nghệ Thông tin — ĐHQG TP. Hồ Chí Minh

Cán bộ hướng dẫn: **TS. Nguyễn Văn Dũ** · **TS. Đỗ Trí Nhựt**

- 1. Đặt vấn đề và mục tiêu nghiên cứu
- 2. Phương pháp thực hiện
- 3. Kết quả 1 — Đánh giá offline loại trừ rò rỉ dữ liệu
- 4. Kết quả 2 — Triển khai phân tán trên cụm thiết bị biên
- 5. Trả lời câu hỏi nghiên cứu và kết luận

Luận văn được phát triển thành 2 bản thảo

FAIR 2026 — so sánh các chiến lược giảm chiều dưới quy trình đánh giá loại trừ rò rỉ (phần kết quả offline).

SOICT 2026 — hệ thống IDS phân tán thời gian thực trên cụm Jetson Orin Nano Super (phần triển khai biên).

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Nội dung trình bày

Nêu rõ ngay từ đầu: luận văn có hai nhánh kết quả, mỗi nhánh đã được viết thành một bản thảo. Điều này giúp hội đồng định vị đóng góp.

Nội dung trình bày	
1. Đặt vấn đề và mục tiêu nghiên cứu 2. Phương pháp thực hiện 3. Kết quả 1 — Đánh giá offline loại trừ rò rỉ dữ liệu 4. Kết quả 2 — Triển khai phân tán trên cụm thiết bị biên 5. Trả lời câu hỏi nghiên cứu và kết luận	Luận văn được phát triển thành 2 bản thảo FAIR 2026 — so sánh các chiến lược giảm chiều dưới quy trình đánh giá loại trừ rò rỉ (phần kết quả offline). SOICT 2026 — hệ thống IDS phân tán thời gian thực trên cụm Jetson Orin Nano Super (phần triển khai biên).

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

└ Đặt vấn đề và mục tiêu nghiên cứu

Đặt vấn đề và mục tiêu nghiên cứu

Đặt vấn đề và mục tiêu nghiên cứu

Bối cảnh và ba khoảng trống nghiên cứu

Vấn đề thực tiễn

- Thiết bị IoT tăng nhanh, tài nguyên ít, bảo mật yếu.
- Lưu lượng lớn, liên tục → cần nền tảng **phân tán**.
- Phát hiện cần đặt **gần nguồn dữ liệu** để giảm băng thông, độ trễ.

Ba khoảng trống

1. RF, SHAP, PCA hiếm khi so sánh trên *cùng một* pipeline phân tán.
2. Điểm số trên CICIDS2017 thường **gần như hoàn hảo** — dấu hiệu rò rỉ dữ liệu.
3. IDS biên thường chỉ đo trên **một thiết bị**.

Câu hỏi trung tâm

Làm sao xây dựng pipeline IDS trên Spark vừa *đánh giá trung thực* ở pha offline, vừa *chạy thời gian thực* trên cụm biên?

Cách tiếp cận: hai pha bổ trợ

- (A) Offline** — mô hình nào, bao nhiêu đặc trưng, dưới quy trình loại trừ rò rỉ.
- (B) Biên** — mô hình đó chạy được thời gian thực không?

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

└ Đặt vấn đề và mục tiêu nghiên cứu

Bối cảnh và ba khoảng trống nghiên cứu

Nhấn mạnh: hai pha trả lời hai câu hỏi khác nhau, không phải hai phương án cạnh tranh.

<

Mục tiêu và ba câu hỏi nghiên cứu

Mục tiêu. Xây dựng hệ thống IDS kết hợp học máy với Apache Spark, phát hiện các dạng tấn công phổ biến trên CICIDS2017; thiết kế pipeline xử lý phân tán; đánh giá khả năng triển khai trên thiết bị biên; áp dụng quy trình đánh giá **loại trừ rò rỉ dữ liệu** có kiểm định thống kê.

RQ1 — Giảm chiều

Giảm chiều/chọn đặc trưng có giữ được hiệu năng gần mức dùng toàn bộ đặc trưng không, khi cắt hơn 60% số đặc trưng?

RQ2 — Học tổ hợp

Học tổ hợp có lợi thế *đáng kể* so với mô hình đơn tốt nhất không, khi được *kiểm định thống kê*?

RQ3 — Khả thi ở biên

Pipeline Spark có khả thi về thông lượng, độ trễ, năng lượng khi triển khai phân tán trên cụm Jetson không?

Ba câu hỏi này được trả lời trực tiếp ở slide kết luận.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Đặt vấn đề và mục tiêu nghiên cứu

Mục tiêu và ba câu hỏi nghiên cứu

Đọc chậm ba RQ. Hội đồng sẽ đối chiếu chúng với phần kết luận.

Mục tiêu và ba câu hỏi nghiên cứu		
Mục tiêu. Xây dựng hệ thống IDS kết hợp học máy với Apache Spark, phát hiện các dạng tấn công phổ biến trên CICIDS2017; thiết kế pipeline xử lý phân tán; đánh giá khả năng triển khai trên thiết bị biên; áp dụng quy trình đánh giá loại trừ rò rỉ dữ liệu có kiểm định thống kê.		
RQ1 — Giảm chiều Giảm chiều/chọn đặc trưng có giữ được hiệu năng gần mức dùng toàn bộ đặc trưng không, khi cắt hơn 60% số đặc trưng?	RQ2 — Học tổ hợp Học tổ hợp có lợi thế đáng kể so với mô hình đơn tốt nhất không, khi được kiểm định thống kê?	RQ3 — Khả thi ở biên Pipeline Spark có khả thi về thông lượng, độ trễ, năng lượng khi triển khai phân tán trên cụm Jetson không?
Ba câu hỏi này được trả lời trực tiếp ở slide kết luận.		

Phạm vi

- **Dữ liệu:** CICIDS2017; mở rộng sang CSE-CIC-IDS2018 cho thí nghiệm liên bộ dữ liệu.
- **Công nghệ:** Spark Core/SQL/Streaming/MLlib; Kafka, PostgreSQL, InfluxDB, Grafana.
- **Bài toán:** nhị phân Benign/Attack (có bổ sung đánh giá đa lớp theo loại tấn công).
- **Triển khai:** cụm 2 Jetson Orin Nano Super (8 GB) + máy chủ điều phối.

Lưu ý trung thực về phạm vi “IoT”

CICIDS2017 là lưu lượng doanh nghiệp *mô phỏng*, không chứa giao thức đặc thù IoT (MQTT, CoAP). Tính chất “IoT” nằm ở **kịch bản triển khai**: toàn bộ pipeline suy luận được đo trên *thiết bị biên tài nguyên hạn chế* — lớp gateway trong kiến trúc IoT ba tầng — chứ không ở bản chất lưu lượng huấn luyện.

Mô hình mối đe dọa

Kẻ tấn công ở bên trong hoặc ở biên mạng; phạm vi giới hạn ở các tấn công đã biết kiểu CICIDS2017. Tấn công đối kháng (adversarial/evasion) **nằm ngoài phạm vi** đánh giá.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT
└ Đặt vấn đề và mục tiêu nghiên cứu

Phạm vi, dữ liệu và mô hình mối đe dọa

Chủ động nêu giới hạn phạm vi IoT trước khi hội đồng hỏi. Đây là câu hỏi phản biện gần như chắc chắn sẽ có.

Phạm vi

- **Dữ liệu:** CICIDS2017; mở rộng sang CSE-CIC-IDS2018 cho thí nghiệm liên bộ dữ liệu.
- **Công nghệ:** Spark Core/SQL/Streaming/MLlib; Kafka, PostgreSQL, InfluxDB, Grafana.
- **Bài toán:** nhị phân Benign/Attack (có bổ sung đánh giá đa lớp theo loại tấn công).
- **Triển khai:** cụm 2 Jetson Orin Nano Super (8 GB) + máy chủ điều phối.

Lưu ý trung thực về phạm vi “IoT”

CICIDS2017 là lưu lượng doanh nghiệp mô phỏng, không chứa giao thức đặc thù IoT (MQTT, CoAP). Tính chất “IoT” nằm ở **kịch bản triển khai**: toàn bộ pipeline suy luận được đo trên *thiết bị biên tài nguyên hạn chế* — lớp gateway trong kiến trúc IoT ba tầng — chứ không ở bản chất lưu lượng huấn luyện.

Mô hình mối đe dọa

Kẻ tấn công ở bên trong hoặc ở biên mạng; phạm vi giới hạn ở các tấn công đã biết kiểu CICIDS2017. Tấn công đối kháng (adversarial/evasion) **nằm ngoài phạm vi** đánh giá.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

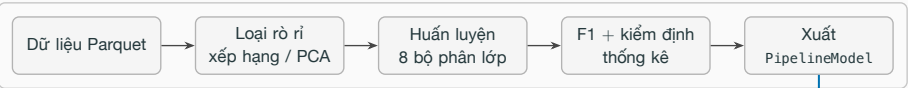
└─ Phương pháp thực hiện

Phương pháp thực hiện

Phương pháp thực hiện

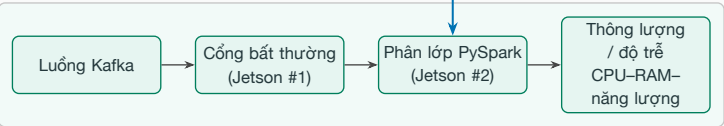
Kiến trúc tổng thể: hai pha của cùng một pipeline

PHA OFFLINE — cụm Spark (máy chủ master + 2 Jetson worker) ⇒ bản thảo FAIR 2026



xuất mô hình đã kiểm định

PHA BIÊN — 2× Jetson Orin Nano Super ⇒ bản thảo SOICT 2026



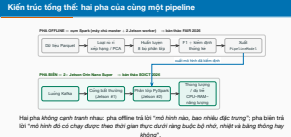
Hai pha không cạnh tranh nhau: pha offline trả lời “mô hình nào, bao nhiêu đặc trưng”; pha biên trả lời “mô hình đó có chạy được theo thời gian thực dưới ràng buộc bộ nhớ, nhiệt và băng thông hay không”

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT
└ Phương pháp thực hiện

Kiến trúc tổng thể: hai pha của cùng một pipeline

Đây là slide bản lề của toàn bộ bài trình bày. Chỉ vào mũi tên xanh: mô hình được triển khai chính là mô hình đã qua kiểm định offline.



Hai nguồn rò rỉ, xử lý trước khi chia tập

- 1. **Đặc trưng rò rỉ nhãn:** destination_port mã hóa trực tiếp dịch vụ bị tấn công → mô hình học thuộc ánh xạ cổng → nhãn thay vì học hành vi luồng.
- 2. **Trùng lặp ở mức đặc trưng:** luồng giống hệt nhau trên toàn bộ cột đặc trưng không bị loại bởi phép khử trùng lặp toàn dòng → cùng một mẫu vào cả hai tập.

Nhóm luồng giống hệt nhau nhưng nhãn xung đột bị loại bỏ cả nhóm thay vì giữ lại một dòng tùy cách phân vùng

Kết quả tiền xử lý (tất định)

Nhóm nhãn mâu thuẫn bị loại	270 nhóm
Số dòng bị loại theo	44.260
Đặc trưng số còn lại	77
Huấn luyện / kiểm tra	1,79 tr / 0,45 tr

Chống thiên lệch lựa chọn

- Tách pha thăm dò khỏi pha xác nhận.
- Chọn mô hình theo tập kiểm định tách từ tập huấn luyện.
- Tập kiểm tra không tham gia bước lựa chọn nào.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT
Phương pháp thực hiện

Pha A — Quy trình đánh giá loại trừ rò rỉ dữ liệu

Đây là đóng góp phương pháp luận quan trọng nhất. Nói rõ vì sao khử trùng lặp tất định lại quan trọng: kết quả không phụ thuộc cách Spark phân vùng dữ liệu.

Pha A — Quy trình đánh giá loại trừ rò rỉ dữ liệu		
Hai nguồn rò rỉ, xử lý trước khi chia tập		Kết quả tiền xử lý (tất định)
1. Đặc trưng rò rỉ nhãn: destination_port mã hóa trực tiếp dịch vụ bị tấn công → mô hình học thuộc ánh xạ cổng → nhãn thay vì học hành vi luồng.	Nhóm nhãn mâu thuẫn bị loại	270 nhóm
	Số dòng bị loại theo	44.260
	Đặc trưng số còn lại	77
	Huấn luyện / kiểm tra	1,79 tr / 0,45 tr
2. Trùng lặp ở mức đặc trưng: luồng giống hệt nhau trên toàn bộ cột đặc trưng không bị loại bởi phép khử trùng lặp toàn dòng → cùng một mẫu vào cả hai tập. Nhóm luồng giống hệt nhau nhưng nhãn xung đột bị loại bỏ cả nhóm, thay vì giữ lại một dòng tùy cách phân vùng.	Chống thiên lệch lựa chọn	
	■ Tách pha thăm dò khỏi pha xác nhận.	
	■ Chọn mô hình theo tập kiểm định tách từ tập huấn luyện.	
	■ Tập kiểm tra không tham gia bước lựa chọn nào.	

Pha A — Bốn chiến lược giảm chiều và các mô hình so sánh

Bốn chiến lược, cùng một pipeline Spark

- **Baseline**: toàn bộ 77 đặc trưng.
- **RF Importance Top-K**: xếp hạng nhúng từ Random Forest.
- **SHAP Top-K**: giá trị Shapley từ XGBoost, tính trên mẫu **phân tầng theo lớp**.
- **PCA k=40**: trích xuất không giám sát.

Xếp hạng RF/SHAP chỉ dùng tập huấn luyện. Lý do PCA dùng k=40: xem slide dự phòng.

8 thuật toán + 2 mô hình học tổ hợp

- Decision Tree, Logistic Regression, SVM, Naive Bayes, Random Forest, GBT, XGBoost, MLP.
- **Soft Voting** và **Hybrid Bagging** — kết hợp Top-3 thuật toán có F1 cao nhất *trên tập kiểm định*.

Mất cân bằng lớp — xử lý thống nhất

- Trọng số lớp cho 6 mô hình; scale_pos_weight cho XGBoost; MLP không hỗ trợ.

2026-08-15

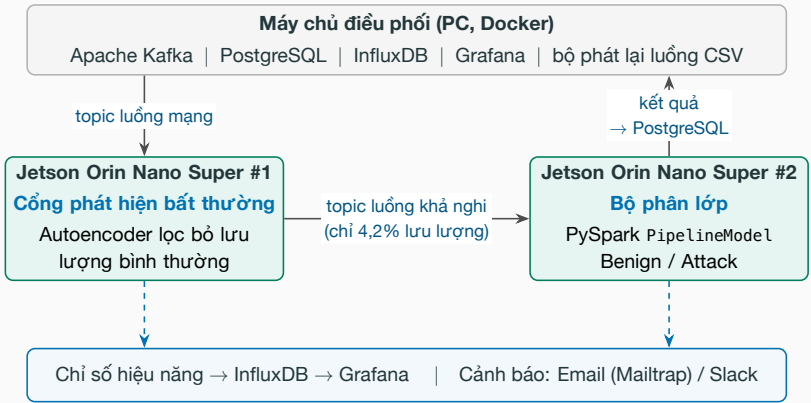
Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT
└─ Phương pháp thực hiện

Pha A — Bốn chiến lược giảm chiều và các mô hình so sánh

Giải thích PCA k=40 chủ động nếu còn thời gian — đây là câu hỏi phản biện hay gặp về tính công bằng của so sánh; slide dự phòng có đầy đủ lý do. LightGBM bị loại vì thư viện native chỉ chạy x86_64, không chạy trên ARM64.

Pha A — Bốn chiến lược giảm chiều và các mô hình so sánh	
Bốn chiến lược, cùng một pipeline Spark ■ Baseline: toàn bộ 77 đặc trưng. ■ RF Importance Top-K: xếp hạng nhúng từ Random Forest. ■ SHAP Top-K: giá trị Shapley từ XGBoost, tính trên mẫu phân tầng theo lớp. ■ PCA k=40: trích xuất không giám sát. Xếp hạng RF/SHAP chỉ dùng tập huấn luyện. Lý do PCA dùng k=40: xem slide dự phòng.	8 thuật toán + 2 mô hình học tổ hợp ■ Decision Tree, Logistic Regression, SVM, Naive Bayes, Random Forest, GBT, XGBoost, MLP. ■ Soft Voting và Hybrid Bagging — kết hợp Top-3 thuật toán có F1 cao nhất trên tập kiểm định. Mất cân bằng lớp — xử lý thống nhất ■ Trọng số lớp cho 6 mô hình; scale_pos_weight cho XGBoost; MLP không hỗ trợ.

Pha B — Kiến trúc triển khai phân tán trên cụm biên

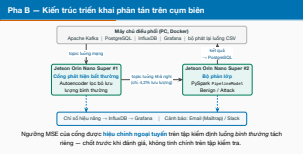


Ngưỡng MSE của cổng được **hiệu chỉnh ngoại tuyến** trên tập kiểm định *lượng bình thường* tách riêng — chốt *trước* khi đánh giá, không tinh chỉnh trên tập kiểm tra.

2026-08-15 Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT
└─ Phương pháp thực hiện

Pha B — Kiến trúc triển khai phân tán trên cụm biên

Nhấn: ngưỡng cổng không được tinh chỉnh trên tập kiểm tra.



Pha B — Ba chế độ triển khai và môi trường đo

Đơn nút toàn bộ pipeline trong một tiến trình — mốc tham chiếu.

A: Tách pipeline cổng trên #1, phân lớp trên #2 (**thiết kế đề xuất**).

B: Mở rộng ngang cả hai nút chạy *vai trò đầy đủ*, chia tải qua Kafka consumer group.

C: Cụm Spark máy chủ làm master, hai Jetson làm worker.

Thông lượng tính theo **số phán quyết cuối cùng/giây**
→ luồng được quyết định tại cổng hoặc tại bộ phân lớp
→ nên luồng chuyển tiếp không bị đếm hai lần

Môi trường thực nghiệm

2× Jetson Orin Nano Super: CPU 6 nhân Arm Cortex-A78AE, GPU Ampere ≈67 TOPS, 8 GB LPDDR5, SSD NVMe 256 GB.

PySpark 3.4.1, Kafka 3.5, LAN Wi-Fi.

Tải: phát lại CICIDS2017 ở 100 luồng/giây.

≥3 lần lặp; lưu lượng khởi động *loại khỏi* cửa sổ đo.

Năng lượng: tegrastats, trừ nền idle.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT
└─ Phương pháp thực hiện

Pha B — Ba chế độ triển khai và môi trường đo

Định nghĩa “phán quyết/giây” là điểm mấu chốt để so sánh công bằng giữa các chế độ.

Pha B — Ba chế độ triển khai và môi trường đo	
Đơn nút toàn bộ pipeline trong một tiến trình — mốc tham chiếu. A: Tách pipeline cổng trên #1, phân lớp trên #2 (thiết kế đề xuất). B: Mở rộng ngang cả hai nút chạy vai trò đầy đủ, chia tải qua Kafka consumer group. C: Cụm Spark máy chủ làm master, hai Jetson làm worker. Thông lượng tính theo số phán quyết cuối cùng/giây → luồng được quyết định tại cổng hoặc tại bộ phân lớp → nên luồng chuyển tiếp không bị đếm hai lần.	Môi trường thực nghiệm 2× Jetson Orin Nano Super: CPU 6 nhân Arm Cortex-A78AE, GPU Ampere ≈67 TOPS, 8 GB LPDDR5, SSD NVMe 256 GB. PySpark 3.4.1, Kafka 3.5, LAN Wi-Fi. Tải: phát lại CICIDS2017 ở 100 luồng/giây. ≥3 lần lặp; lưu lượng khởi động loại khỏi cửa sổ đo. Năng lượng: tegrastats, trừ nền idle.

Kết quả 1 — Đánh giá offline loại trừ rò rỉ

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

└─Kết quả 1 — Đánh giá offline loại trừ rò rỉ

Kết quả 1 — Đánh giá offline loại
trừ rò rỉ

KQ1 — Giảm 60% đặc trưng mà vẫn giữ hiệu năng

Phương pháp	Mô hình	F1	Huấn luyện (s)	Đặc trưng
Baseline	XGBoost	0.9971	4615.5	77
SHAP Top-30	XGBoost	0,9970	1226,1	30
PCA-k=40	XGBoost	0.9959	1141.3	30
RF Top-30	XGBoost	0.9922	1295.1	30

Thuật toán	RF Top-30	SHAP Top-30	Chênh lệch
Random Forest	0.9879	0.9955	+0,77%
XGBoost	0.9922	0.9970	+0,48%
GBT	0.9844	0.9918	+0,75%
Decision Tree	0.9835	0.9948	+1,15%

- SHAP Top-30 giữ F1 ngang baseline, cắt **≈60% đặc trưng** và **≈73% thời gian huấn luyện**.
- Ở cùng số đặc trưng, SHAP vượt RF Importance trên mọi thuật toán họ cây.
- Học tổ hợp **Hybrid Bagging** với SHAP Top-30 đạt F1 **0,9966** — cao nhất trong các cấu hình đã thử.
- Ít đặc trưng hơn → chi phí VectorAssembler và bộ nhớ mô hình ở biên nhỏ hơn.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

— Kết quả 1 — Đánh giá offline loại trừ rò rỉ

KQ1 — Giảm 60% đặc trưng mà vẫn giữ hiệu năng

Trả lời RQ1 ngay tại đây, nhưng chưa kết luận vội — còn phải kiểm định thống kê ở slide sau. Nói thêm: SHAP còn cho phép giải thích từng quyết định, có giá trị cho vận hành SOC.

Phương pháp	Mô hình	F1	Huấn luyện (s)	Đặc trưng
Baseline	XGBoost	0.9971	4615.5	77
SHAP Top-30	XGBoost	0,9970	1226,1	30
PCA-k=40	XGBoost	0.9959	1141.3	30
RF Top-30	XGBoost	0.9922	1295.1	30

Thuật toán	RF Top-30	SHAP Top-30	Chênh lệch
Random Forest	0.9879	0.9955	+0,77%
XGBoost	0.9922	0.9970	+0,48%
GBT	0.9844	0.9918	+0,75%
Decision Tree	0.9835	0.9948	+1,15%

- SHAP Top-30 giữ F1 ngang baseline, cắt **≈60% đặc trưng** và **≈73% thời gian huấn luyện**.
- Ở cùng số đặc trưng, SHAP vượt RF Importance trên mọi thuật toán họ cây.
- Học tổ hợp **Hybrid Bagging** với SHAP Top-30 đạt F1 **0,9966** — cao nhất trong các cấu hình đã thử.
- Ít đặc trưng hơn → chi phí VectorAssembler và bộ nhớ mô hình ở biên nhỏ hơn.

KQ2 — Kiểm định thống kê thay vì so sánh một lần đo

Phương pháp	Mô hình	F1 (TB ± CI95)
Baseline	XGBoost	0.99734 ± 0.00007
SHAP Top-30	XGBoost	0.99720 ± 0.00009
hoán vị ghép cặp $p = 0.031$, $d = 1.94$		
Ablation công (RF)	F1	Recall (Attack)
Giữ destruction part	0.9955	0.9988
Loại công (để xuất)	0.9955	0.9961

- $N=6$ lần *lấy mẫu lại* ghép cặp → phương sai phản ánh tác động của lấy mẫu dữ liệu, không chỉ của seed.
- Kiểm định hoán vị hai phía **liệt kê đầy đủ** 2^6 hoán vị dấu: p nhỏ nhất là $2/2^6$, nên **bắt buộc $N \geq 6$** .
- Các lần lấy mẫu *chống lén* nên vi phạm giả định độc lập của phân phối $t \rightarrow$ kết luận theo khoảng tin cậy hiệu chỉnh **Nadeau-Bengio**.

Khác biệt có ý nghĩa *chỉ ở ngưỡng* sàn của độ phân giải kiểm định, chênh lệch tuyệt đối $\approx 0,0001 \rightarrow$ hai cấu hình **tương đương về mặt thực tiễn**; tiêu chí phụ mới quyết định.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT
└─ Kết quả 1 — Đánh giá offline loại trừ rò rỉ

KQ2 — Kiểm định thống kê thay vì so sánh một lần đo

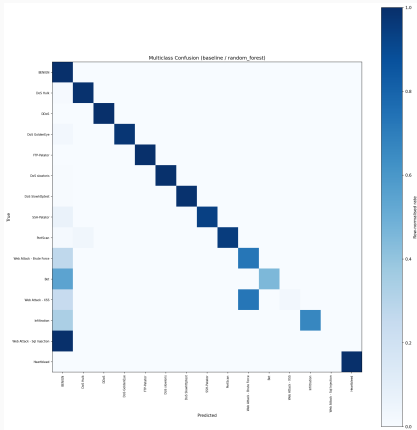
Đây là chỗ thể hiện sự cẩn trọng phương pháp: p nhỏ không đồng nghĩa khác biệt có ý nghĩa thực tiễn.

Phương pháp	Mô hình	F1 (TB ± CI95)
Baseline	XGBoost	0.99734 ± 0.00007
SHAP Top-30	XGBoost	0.99720 ± 0.00009
hoán vị ghép cặp $p = 0.031$, $d = 1.94$		
Ablation công (RF)	F1	Recall (Attack)
Giữ destruction part	0.9955	0.9988
Loại công (để xuất)	0.9955	0.9961

Khác biệt có ý nghĩa chỉ ở ngưỡng sàn của độ phân giải kiểm định, chênh lệch tuyệt đối $\approx 0,0001 \rightarrow$ hai cấu hình *tương đương về mặt thực tiễn*, tiêu chí phụ mới quyết định.

- $N=6$ lần lấy mẫu lại ghép cặp $\rightarrow$ phương sai phản ánh tác động của lấy mẫu dữ liệu, không chỉ của seed.
- Kiểm định hoán vị hai phía **liệt kê đầy đủ** 2^6 hoán vị dấu: p nhỏ nhất là $2/2^6$, nên **bắt buộc $N \geq 6$** .
- Các lần lấy mẫu chống lén nên vi phạm giả định độc lập của phân phối $t \rightarrow$ kết luận theo khoảng tin cậy hiệu chỉnh **Nadeau-Bengio**.

KQ3 – Đánh giá đa lớp: F1 nhị phân che giấu điều quan trọng



Lớp	Số mẫu	Recall	F1
Benign	380.119	0.9997	0.9989
DoS Hulk	34.446	0.9904	0.9947
Web – Brute Force	311	0.7299	0.7323
Bot	290	0,4552	0.6256
Web – XSS	111	0,0270	0.0526
Web – SQL Injection	6	0,0000	0.0000
weighted-F1 = 0.998		macro-F1 = 0,806	

- **Nhầm loại nhưng vẫn bắt được:** 81/111 XSS bị đoán thành Web Brute Force → **76% vẫn bị gắn cờ** ở mức nghi phân.
- **Lọt hỏn thành Benign** (nguy hiỏm): cả 6 mẫu SQL Injection, **54% Bot**, 27% Brute Force.

8-15 Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Kết quả 1 – Đánh giá offline loại trừ rò rỉ



Ba cơ chế: mất cân bằng cực đoan (6–311 mẫu so với 380 nghìn benign); loại `destination_port` tác động mạnh nhất lên web attack — mất lỗi tắt cổng 80, luồng HTTP ngắn trông như duyệt web bình thường, nên recall 0,027 trung thực thay thế con số bị rò rỉ thời phỏng; và beaconing C&C của Bot vốn giống lưu lượng nền.

KQ4 — Tổng quát hóa liên bộ dữ liệu: con số trung thực nhất

Huấn luyện	Kiểm tra	F1
CICIDS2017	CICIDS2017	0,9952
CICIDS2017	CSE-CIC-IDS2018	0,0423
CSE-CIC-IDS2018	CSE-CIC-IDS2018	0,9245
CSE-CIC-IDS2018	CICIDS2017	0,0535

Tập đặc trưng chung đã loại bỏ rỏ rĩ
CSE-CIC-IDS2018 lấy mẫu phân tầng theo
nhân (~2,39 triệu luồng, seed cố định).

Điểm số trong-miền gần hoàn hảo — kể cả khi đã loại rò rỉ — **không chứng nhận** khả năng triển khai dưới dịch chuyển phân phối.

- F1 trong-miền cao ở cả hai chiều — nhưng F1 liên bộ dữ liệu sụp xuống **0,04–0,05**.
- Do recall gần 0 (0,022 và 0,030) trong khi precision chiều 2017→2018 vẫn đạt 0,596: mô hình **không nhận ra** tấn công của testbed kia, chứ không báo động bừa bãi.
- Nguyên nhân: khác phiên bản CICFlowMeter, khác cấu hình mạng, bộ công cụ tấn công không trùng nhau.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

— Kết quả 1 — Đánh giá offline loại trừ rò rỉ

KQ4 — Tổng quát hóa liên bộ dữ liệu: con số trung thực nhất

Không làm nhẹ con số này. Nó thống nhất với các nghiên cứu liên bộ dữ liệu trước đó và là lý do thích ứng miền là điều kiện tiên quyết.

KQ4 — Tổng quát hóa liên bộ dữ liệu: con số trung thực nhất

Huấn luyện	Kiểm tra	F1
CICIDS2017	CICIDS2017	0,9952
CICIDS2017	CSE-CIC-IDS2018	0,0423
CSE-CIC-IDS2018	CSE-CIC-IDS2018	0,9245
CSE-CIC-IDS2018	CICIDS2017	0,0535

Tập đặc trưng chung đã loại bỏ rỏ rĩ;
CSE-CIC-IDS2018 lấy mẫu phân tầng theo nhân (~2,39 triệu luồng, seed cố định).

■ F1 trong-miền cao ở cả hai chiều — nhưng F1 liên bộ dữ liệu sụp xuống **0,04–0,05**.

■ Do recall gần 0 (0,022 và 0,030) trong khi precision chiều 2017→2018 vẫn đạt 0,596: mô hình không nhận ra tấn công của testbed kia, chứ không báo động bừa bãi.

■ Nguyên nhân: khác phiên bản CICFlowMeter, khác cấu hình mạng, bộ công cụ tấn công không trùng nhau.

Điểm số trong-miền gần hoàn hảo — kể cả khi đã loại rò rỉ — **không chứng nhận** khả năng triển khai dưới dịch chuyển phân phối.

13/21

Kết quả 2 — Triển khai phân tán trên cụm biên

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

- └─ Kết quả 2 — Triển khai phân tán trên cụm biên

Kết quả 2 — Triển khai phân tán
trên cụm biên

KQ5 — Tách pipeline nâng thông lượng gấp 24 lần

Chế độ	Thông lượng (luồng/giây)	Độ trễ p95 (ms)	Bộ qua tải công (%)	Năng lượng (mJ/phán quyết)
Đơn nút	2,80 ± 0,41	12,3 ± 3,0	(nội tuyến)	2490
A: Tách pipeline	62,5 ± 0,6	13,1 ± 2,6	95,8	178
B: Mở rộng ngang	5,9 ± 1,4	21,6 ± 19,2	(nội tuyến)	2250
C: Cụm Spark	90,0 ± 25,6	23,9 ± 17,8	97,9	119

- **A: gấp 24 lần, p95 gần như không đổi** — cổng hấp thụ 95,8% lưu lượng bình thường.

■ **B vẫn bị chặn bởi Spark** (≈2 lần): lợi ích đến từ *tách vai trò*, không phải từ thêm board.
- C nhanh nhất về trung bình nhưng **±26 luồng/giây** và phụ thuộc Spark master.

■ Năng lượng phân bổ trên nhiều phán quyết hơn 24–35 lần: **2,5 → 0,12–0,18 J**.

Tách pipeline là lựa chọn mặc định được khuyến nghị: thông lượng cao nhất trên mỗi đơn vị phương sai.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

— Kết quả 2 — Triển khai phân tán trên cụm biên

KQ5 — Tách pipeline nâng thông lượng gấp 24 lần

Bảng cốt lõi của pha B. Chỉ rõ chế độ B là bằng chứng phản chứng. Năng lượng là mức toàn board trên mỗi phán quyết: ở tải này phần công suất tích cực nằm dưới ngưỡng nhiễu của tegrastats.

KQ5 — Tách pipeline nâng thông lượng gấp 24 lần				
Chế độ	Thông lượng (luồng/giây)	Độ trễ p95 (ms)	Bộ qua tải công (%)	Năng lượng (mJ/phán quyết)
Đơn nút	2,80 ± 0,41	12,3 ± 3,0	(nội tuyến)	2490
A: Tách pipeline	62,5 ± 0,6	13,1 ± 2,6	95,8	178
B: Mở rộng ngang	5,9 ± 1,4	21,6 ± 19,2	(nội tuyến)	2250
C: Cụm Spark	90,0 ± 25,6	23,9 ± 17,8	97,9	119

■ **A: gấp 24 lần, p95 gần như không đổi** — cổng hấp thụ 95,8% lưu lượng bình thường.

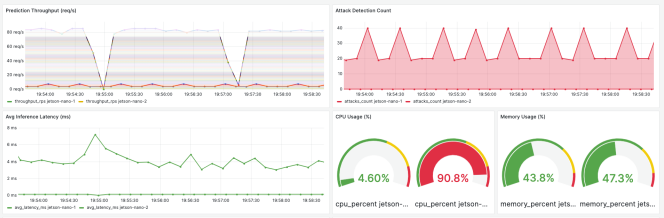
■ **B vẫn bị chặn bởi Spark** (≈2 lần): lợi ích đến từ *tách vai trò*, không phải từ thêm board.

■ **C nhanh nhất về trung bình nhưng ±26 luồng/giây** và phụ thuộc Spark master.

■ **Năng lượng phân bổ trên nhiều phán quyết hơn 24–35 lần: 2,5 → 0,12–0,18 J**.

Tách pipeline là lựa chọn mặc định được khuyến nghị: thông lượng cao nhất trên mỗi đơn vị phương sai.

KQ6 — Bất đối xứng tải quan sát được trực tiếp



- nút cổng: **4,6% CPU**
- nút phân lớp: **90,8% CPU**

Bộ phân lớp chiếm gần như toàn bộ ngân sách tính toán — chính vì vậy việc đưa nó ra khỏi đường ingest mới mang lại hiệu quả.

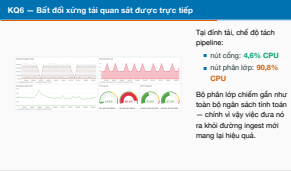
2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

└─ Kết quả 2 — Triển khai phân tán trên cụm biên

KQ6 — Bất đối xứng tải quan sát được trực tiếp

Ảnh chụp Grafana thật, không phải sơ đồ. Hệ thống giám sát thời gian thực: thông lượng, số tấn công phát hiện, độ trễ, CPU/RAM từng nút, cảnh báo lưu trong PostgreSQL.



Luận văn phục vụ suy luận bằng Spark để *đồng nhất với pha huấn luyện phân tán*. Cùng mô hình rừng ngẫu nhiên, cùng 29 đặc trưng, cùng một board:

Engine suy luận	Thông lượng (lượt/giây)	p95 (ms)	RAM (%)	Năng lượng (mJ/suy luận)
PySpark (đang triển khai)	22,6	532	24,3	71,6
scikit-learn	204,4	49,8	13,3	2,91
ONNX Runtime	7870,8	1,1	15,7	0,06

ONNX Runtime đạt thông lượng gấp **≈348 lần** bản PySpark đang triển khai. Luận văn báo cáo con số này như một *mục tiêu cho bước tiếp theo* — xuất mô hình sang ONNX/TensorRT cho pha phục vụ, giữ Spark cho pha huấn luyện — chứ không phải lập luận bác bỏ stack đồng nhất.

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

— Kết quả 2 — Triển khai phân tán trên cụm biên

KQ7 — Cái giá trung thực của việc đồng nhất một stack

Chủ động trình bày điểm yếu này. Hội đồng sẽ đánh giá cao việc tự định lượng chi phí của lựa chọn thiết kế thay vì che giấu.

2026-08-15

KQ7 — Cái giá trung thực của việc đồng nhất một stack				
Luận văn phục vụ suy luận bằng Spark để đồng nhất với pha huấn luyện phân tán. Cùng mô hình rừng ngẫu nhiên, cùng 29 đặc trưng, cùng một board:				
Engine suy luận	Thông lượng (lượt/giây)	p95 (ms)	RAM (%)	Năng lượng (mJ/suy luận)
PySpark (đang triển khai)	22,6	532	24,3	71,6
scikit-learn	204,4	49,8	13,3	2,91
ONNX Runtime	7870,8	1,1	15,7	0,06

ONNX Runtime đạt thông lượng gấp **≈348 lần** bản PySpark đang triển khai. Luận văn báo cáo con số này như một mục tiêu cho bước tiếp theo — xuất mô hình sang ONNX/TensorRT cho pha phục vụ, giữ Spark cho pha huấn luyện — chứ không phải lập luận bác bỏ stack đồng nhất.

Kết luận

Trả lời trực tiếp ba câu hỏi nghiên cứu

RQ1 — Giảm chiều có giữ được hiệu năng khi cắt hơn 60% đặc trưng?

Có. SHAP Top-30 đạt F1 0,9970 (XGBoost) / 0,9955 (Random Forest) so với 0,9971 của baseline 77 đặc trưng, đồng thời giảm $\approx 73\%$ thời gian huấn luyện. Ở cùng số đặc trưng, SHAP vượt RF Importance trên mọi thuật toán họ cây (+0,48% đến +1,15%).

RQ2 — Học tổ hợp có lợi thế đáng kể so với mô hình đơn tốt nhất?

Không đáng kể về mặt thực tiễn. Hybrid Bagging (SHAP Top-30) đạt F1 0,9966 — cao nhất — nhưng kiểm định hoán vị ghép cặp cho thấy khác biệt giữa các cấu hình hàng đầu chỉ có ý nghĩa ở ngưỡng sàn ($p \approx 0,031$) với chênh lệch tuyệt đối $\approx 0,0001$.

RQ3 — Pipeline Spark có khả thi trên cụm thiết bị biên?

Khả thi, với điều kiện tách vai trò. Tách pipeline nâng thông lượng từ 2,6 lên 62,5 phán quyết/giây ở p95 ≈ 13 ms; năng lượng giảm từ $\approx 2,5$ J xuống 0,18 J mỗi phán quyết. Chỉ thêm board mà không tách vai trò (chế độ B) hầu như không cải thiện.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

— Kết luận

Trả lời trực tiếp ba câu hỏi nghiên cứu

Slide quan trọng nhất với hội đồng. Trả lời thẳng, kể cả câu trả lời “không” cho RQ2. Bổ sung cho RQ2: bài toán nhị phân trong-miền đã gần bão hòa, nên tiêu chí phụ mới là căn cứ chọn mô hình.

Trả lời trực tiếp ba câu hỏi nghiên cứu	
RQ1 — Giảm chiều có giữ được hiệu năng khi cắt hơn 60% đặc trưng?	Có. SHAP Top-30 đạt F1 0,9970 (XGBoost) / 0,9955 (Random Forest) so với 0,9971 của baseline 77 đặc trưng, đồng thời giảm $\approx 73\%$ thời gian huấn luyện. Ở cùng số đặc trưng, SHAP vượt RF Importance trên mọi thuật toán họ cây (+0,48% đến +1,15%).
RQ2 — Học tổ hợp có lợi thế đáng kể so với mô hình đơn tốt nhất?	Không đáng kể về mặt thực tiễn. Hybrid Bagging (SHAP Top-30) đạt F1 0,9966 — cao nhất — nhưng kiểm định hoán vị ghép cặp cho thấy khác biệt giữa các cấu hình hàng đầu chỉ có ý nghĩa ở ngưỡng sàn ($p \approx 0,031$) với chênh lệch tuyệt đối $\approx 0,0001$.
RQ3 — Pipeline Spark có khả thi trên cụm thiết bị biên?	Khả thi, với điều kiện tách vai trò. Tách pipeline nâng thông lượng từ 2,6 lên 62,5 phán quyết/giây ở p95 ≈ 13 ms; năng lượng giảm từ $\approx 2,5$ J xuống 0,18 J mỗi phán quyết. Chỉ thêm board mà không tách vai trò (chế độ B) hầu như không cải thiện.

Về phương pháp luận

- **Quy trình đánh giá loại trừ rò rỉ:** xử lý hai nguồn rò rỉ, khử trùng lặp *tất định* ở mức đặc trưng, định lượng mức thổi phồng do rò rỉ.
- **Kiểm định thống kê chặt chẽ:** hoán vị ghép cặp liệt kê đầy đủ, khoảng tin cậy Nadeau–Bengio, kích thước hiệu ứng.
- **Đánh giá đa lớp và liên bộ dữ liệu:** chỉ ra điều F1 nhị phân che giấu (macro-F1 0,806) và giới hạn tổng quát hóa.

Về hệ thống

- **Pipeline IDS hoàn chỉnh trên Apache Spark:** từ tiền xử lý, trích chọn đặc trưng, huấn luyện phân tán đến suy luận thời gian thực.
- **Kiến trúc IDS hai tầng trên cụm biên:** cổng bất thường + phân lớp PySpark tách trên hai board, nối bằng Kafka; ba chế độ triển khai được đo.
- **Đo đặc vận hành đầy đủ:** thông lượng, phân vị độ trễ, CPU/RAM, năng lượng mỗi phán quyết.
- **Mã nguồn và cấu hình công khai,** tái lập được.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

— Kết luận

Đóng góp chính của luận văn

Chia rõ hai nhóm đóng góp: phương pháp luận và hệ thống.

Đóng góp chính của luận văn	
Về phương pháp luận ■ Quy trình đánh giá loại trừ rò rỉ: xử lý hai nguồn rò rỉ, khử trùng lặp <i>tất định</i> ở mức đặc trưng, định lượng mức thổi phồng do rò rỉ. ■ Kiểm định thống kê chặt chẽ: hoán vị ghép cặp liệt kê đầy đủ, khoảng tin cậy Nadeau–Bengio, kích thước hiệu ứng. ■ Đánh giá đa lớp và liên bộ dữ liệu: chỉ ra điều F1 nhị phân che giấu (macro-F1 0,806) và giới hạn tổng quát hóa.	Về hệ thống ■ Pipeline IDS hoàn chỉnh trên Apache Spark: từ tiền xử lý, trích chọn đặc trưng, huấn luyện phân tán đến suy luận thời gian thực. ■ Kiến trúc IDS hai tầng trên cụm biên: cổng bất thường + phân lớp PySpark tách trên hai board, nối bằng Kafka; ba chế độ triển khai được đo. ■ Đo đặc vận hành đầy đủ: thông lượng, phân vị độ trễ, CPU/RAM, năng lượng mỗi phán quyết. ■ Mã nguồn và cấu hình công khai, tái lập được.

- **Phạm vi phân loại.** Triển khai chính là bài toán nhị phân; luận văn đã bổ sung đánh giá đa lớp để phân tích lớp hiếm, nhưng một bộ phân lớp đa lớp hoàn chỉnh cho thời gian thực vẫn còn ở phía trước.
- **Phạm vi dữ liệu.** CICIDS2017 không chứa lưu lượng IoT đặc thù; thí nghiệm liên bộ dữ liệu mới thực hiện trên *mẫu phân tầng* của CSE-CIC-IDS2018, chưa phải toàn bộ.
- **Quy mô.** Khía cạnh xử lý phân tán mới được kiểm chứng ở mức *minh chứng khái niệm* trên cụm hai nút 8 GB với vài triệu luồng, chưa phải quy mô terabyte hay cụm nhiều nút mạnh.
- **Đối kháng và tài nguyên chưa khai thác.** Chưa kiểm thử trước lưu lượng né tránh (adversarial/evasion); GPU ≈ 67 TOPS còn bỏ trống do pipeline hiện chỉ dùng CPU.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

— Kết luận

Hạn chế của đề tài

Nêu hạn chế trước khi hội đồng nêu. Mỗi hạn chế có một hướng khắc phục tương ứng ở slide sau.

Hạn chế của đề tài

- **Phạm vi phân loại.** Triển khai chính là bài toán nhị phân; luận văn đã bổ sung đánh giá đa lớp để phân tích lớp hiếm, nhưng một bộ phân lớp đa lớp hoàn chỉnh cho thời gian thực vẫn còn ở phía trước.
- **Phạm vi dữ liệu.** CICIDS2017 không chứa lưu lượng IoT đặc thù; thí nghiệm liên bộ dữ liệu mới thực hiện trên *mẫu phân tầng* của CSE-CIC-IDS2018, chưa phải toàn bộ.
- **Quy mô.** Khía cạnh xử lý phân tán mới được kiểm chứng ở mức *minh chứng khái niệm* trên cụm hai nút 8 GB với vài triệu luồng, chưa phải quy mô terabyte hay cụm nhiều nút mạnh.
- **Đối kháng và tài nguyên chưa khai thác.** Chưa kiểm thử trước lưu lượng né tránh (adversarial/evasion); GPU ≈ 67 TOPS còn bỏ trống do pipeline hiện chỉ dùng CPU.

- **Cải thiện độ nhạy cho lớp tấn công hiểm.** Xử lý mất cân bằng có chủ đích (undersampling/SMOTE áp dụng *sau* khi chia tập), focal loss, hạ ngưỡng quyết định theo lớp; **kiến trúc phân tầng** — tận dụng chính thiết kế cổng hai tầng: tầng nhị phân ở biên giữ recall tổng, còn bộ định danh loại chỉ huấn luyện trên *luồng tấn công*, huấn luyện lại ngoại tuyến mà không đụng hệ thống đang chạy; thêm **đặc trưng chuỗi thời gian theo host** cho beaconing C&C.
- **Kiểm chứng trên dữ liệu IoT chuyên biệt.** Bot-IoT, TON_IoT, CIC-IoT-2023; mở rộng thí nghiệm liên bộ dữ liệu lên toàn bộ CSE-CIC-IDS2018.
- **Tối ưu suy luận biên bằng ONNX/TensorRT.** Tích hợp phiên suy luận ONNX vào pipeline streaming, tăng tốc bằng TensorRT trên GPU Jetson; đo độ trễ đầu cuối đầy đủ.
- **Bền vững trước tấn công đối kháng.** Đặc biệt hai bề mặt tấn công mới do cổng tạo ra: giả dạng lưu lượng bình thường để vượt cổng, và cố ý tạo sai số tái tạo cao để dồn tải lên nút phân lớp.

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Kết luận

Hướng phát triển

Mỗi hướng phát triển ứng với một hạn chế ở slide trước — đó là cách thể hiện sự làm chủ đề tài.

Hướng phát triển

- **Cải thiện độ nhạy cho lớp tấn công hiểm.** Xử lý mất cân bằng có chủ đích (undersampling/SMOTE áp dụng *sau* khi chia tập), focal loss, hạ ngưỡng quyết định theo lớp; **kiến trúc phân tầng** — tận dụng chính thiết kế cổng hai tầng: tầng nhị phân ở biên giữ recall tổng, còn bộ định danh loại chỉ huấn luyện trên *luồng tấn công*, huấn luyện lại ngoại tuyến mà không đụng hệ thống đang chạy; thêm **đặc trưng chuỗi thời gian theo host** cho beaconing C&C.
- **Kiểm chứng trên dữ liệu IoT chuyên biệt.** Bot-IoT, TON_IoT, CIC-IoT-2023; mở rộng thí nghiệm liên bộ dữ liệu lên toàn bộ CSE-CIC-IDS2018.
- **Tối ưu suy luận biên bằng ONNX/TensorRT.** Tích hợp phiên suy luận ONNX vào pipeline streaming, tăng tốc bằng TensorRT trên GPU Jetson; đo độ trễ đầu cuối đầy đủ.
- **Bền vững trước tấn công đối kháng.** Đặc biệt hai bề mặt tấn công mới do cổng tạo ra: giả dạng lưu lượng bình thường để vượt cổng, và cố ý tạo sai số tái tạo cao để dồn tải lên nút phân lớp.

1. Thai Nguyen Vu, **Dung Van Bui**, Nhut Tri Do, Van Du Nguyen, “A Comparison of Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML with a Leakage-Aware Evaluation Pipeline”, Hội thảo quốc gia Nghiên cứu cơ bản và ứng dụng CNTT (**FAIR**), 2026.

2. Thai Nguyen Vu, **Dung Van Bui**, Nhut Tri Do, Van Du Nguyen, “A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark”, Symposium on Information and Communication Technology (**SOICT**), 2026.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

— Kết luận

Công trình khoa học từ luận văn

1. Thai Nguyen Vu, **Dung Van Bui**, Nhut Tri Do, Van Du Nguyen, “A Comparison of Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML with a Leakage-Aware Evaluation Pipeline”, Hội thảo quốc gia Nghiên cứu cơ bản và ứng dụng CNTT (**FAIR**), 2026.

2. Thai Nguyen Vu, **Dung Van Bui**, Nhut Tri Do, Van Du Nguyen, “A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark”, Symposium on Information and Communication Technology (**SOICT**), 2026.

Xin trân trọng cảm ơn Hội đồng

Em xin lắng nghe các ý kiến nhận xét và câu hỏi.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

└─ **Kết luận**

Xin trân trọng cảm ơn Hội đồng
Em xin lắng nghe các ý kiến nhận xét và câu hỏi.

Slide dự phòng

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Slide dự phòng

Dự phòng — Siêu tham số cố định trước (a priori)

Mô hình	Siêu tham số chính
Decision Tree	maxDepth=15, minInstancesPerNode=5, impurity=entropy
Logistic Regression	maxIter=200, regParam=0.001, L2, threshold=0.5
SVM (LinearSVC)	maxIter=200, regParam=0.001
Naive Bayes	gaussian, smoothing=1.0
Random Forest	numTrees=200, maxDepth=15, featureSubset=sqrt
GBT	maxIter=150, maxDepth=6, stepSize=0.05, subsample=0.8
XGBoost	n_estimators=300, max_depth=8, lr=0.05, subsample=0.8
MLP	layers=[d,64,32,2], maxIter=150, stepSize=0.01

Dùng chung cho mọi phương pháp giảm chiều, để khác biệt quan sát được phản ánh dùng *phương pháp chọn đặc trưng*. Tối ưu siêu tham số (grid search + cross-validation) tách thành giai đoạn riêng, chỉ áp dụng cho cấu hình đã chọn — mức cải thiện nhỏ, phù hợp với nhận định bài toán nhị phân trong miền đã gán bảo hòa.

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Dự phòng — Siêu tham số cố định trước (a priori)

Dự phòng — Siêu tham số cố định trước (a priori)	
Mô hình	Siêu tham số chính
Decision Tree	maxDepth=15, minInstancesPerNode=5, impurity=entropy
Logistic Regression	maxIter=200, regParam=0.001, L2, threshold=0.5
SVM (LinearSVC)	maxIter=200, regParam=0.001
Naive Bayes	gaussian, smoothing=1.0
Random Forest	numTrees=200, maxDepth=15, featureSubset=sqrt
GBT	maxIter=150, maxDepth=6, stepSize=0.05, subsample=0.8
XGboost	n_estimators=300, max_depth=8, lr=0.05, subsample=0.8
MLP	layers=[d,64,32,2], maxIter=150, stepSize=0.01
Dùng chung cho mọi phương pháp giảm chiều, để khác biệt quan sát được phản ánh dùng phương pháp chọn đặc trưng. Tối ưu siêu tham số (grid search + cross-validation) tách thành giai đoạn riêng, chỉ áp dụng cho cấu hình đã chọn — mức cải thiện nhỏ, phù hợp với nhận định bài toán nhị phân trong miền đã gán bảo hòa.	

PCA $k=40$ thay vì $k=30$

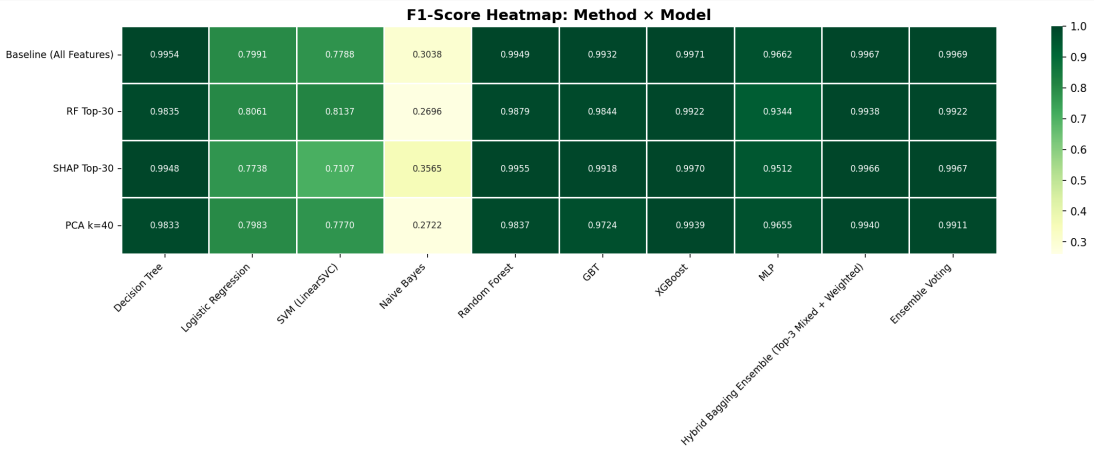
PCA là phép *trích xuất*, không phải *chọn lọc*: mỗi thành phần chính là một tổ hợp tuyến tính chỉ mang một phần phương sai, nên cần nhiều thành phần hơn để giữ lượng thông tin tương đương tập 30 đặc trưng gốc. Trong dải quét chung $\{20, 30, 40\}$, $k=40$ là mức giữ phương sai cao nhất, nên là điểm vận hành đại diện *công bằng nhất* cho PCA; ép $k=30$ sẽ làm PCA mất thêm phương sai và bất lợi khi so sánh.

LightGBM bị loại khỏi so sánh

Thư viện native của LightGBM (qua SynapseML) chỉ hỗ trợ kiến trúc x86_64, không chạy được trên thiết bị biên Jetson Orin Nano Super (ARM64) — vốn là *đích triển khai* của luận văn. Trong họ gradient boosting, XGBoost và GBT đóng vai trò đại diện.

Dự phòng — Vì sao PCA dùng $k=40$, và vì sao loại LightGBM?
PCA $k=40$ thay vì $k=30$ PCA là phép <i>trích xuất</i>, không phải <i>chọn lọc</i>: mỗi thành phần chính là một tổ hợp tuyến tính chỉ mang một phần phương sai, nên cần nhiều thành phần hơn để giữ lượng thông tin tương đương tập 30 đặc trưng gốc. Trong dải quét chung $\{20, 30, 40\}$, $k=40$ là mức giữ phương sai cao nhất, nên là điểm vận hành đại diện công bằng nhất cho PCA; ép $k=30$ sẽ làm PCA mất thêm phương sai và bất lợi khi so sánh. LightGBM bị loại khỏi so sánh Thư viện native của LightGBM (qua SynapseML) chỉ hỗ trợ kiến trúc x86_64, không chạy được trên thiết bị biên Jetson Orin Nano Super (ARM64) — vốn là <i>đích triển khai</i> của luận văn. Trong họ gradient boosting, XGBoost và GBT đóng vai trò đại diện.

Dự phòng — Bản đồ nhiệt F1: phương pháp × thuật toán

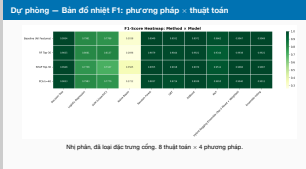


Nhị phân, đã loại đặc trưng cộng. 8 thuật toán × 4 phương pháp.

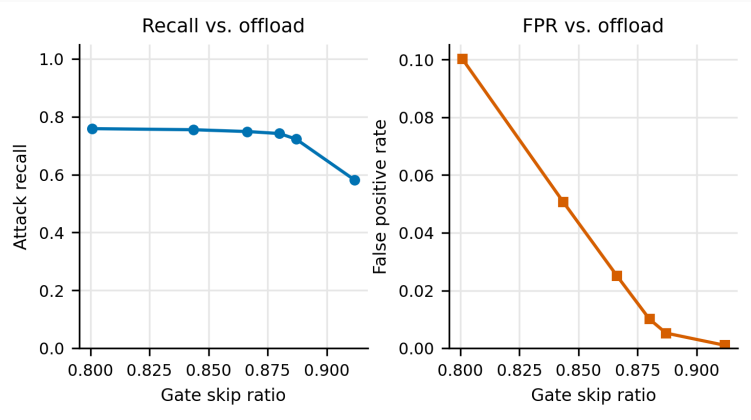
2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Dự phòng — Bản đồ nhiệt F1: phương pháp × thuật toán



Dự phòng — Cổng bất thường là một *đường cong vận hành*

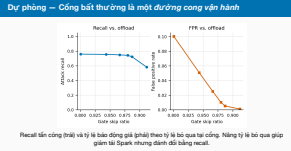


Recall tấn công (trái) và tỷ lệ báo động giả (phải) theo tỷ lệ bỏ qua tại cổng. Năng tỷ lệ bỏ qua giúp giảm tải Spark nhưng đánh đổi bằng recall.

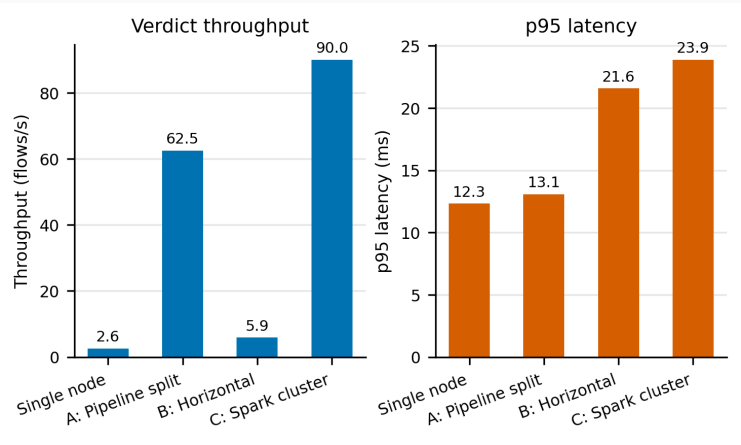
2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Dự phòng — Cổng bất thường là một *đường cong vận*



Dự phòng — So sánh ba chế độ triển khai (biểu đồ)



Thông lượng phân quyết (trái) và độ trễ p95 (phải)

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Dự phòng — So sánh ba chế độ triển khai (biểu đồ)



Dự phòng — Bảng giám sát Grafana đầy đủ

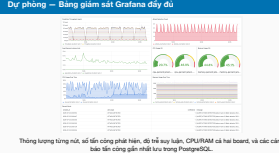


Thông lượng từng nút, số tấn công phát hiện, độ trễ suy luận, CPU/RAM cả hai board, và các cảnh báo tấn công gần nhất lưu trong PostgreSQL.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Dự phòng — Bảng giám sát Grafana đầy đủ



- **Nội tại.** Xếp hạng RF/SHAP lấy từ tập huấn luyện gốc và giữ cố định qua các lần lấy mẫu lại — nhất quán với thiết kế “cấu hình cố định trước”, nhưng có thể gây rò rỉ nhẹ ở phần xếp hạng. Xếp hạng lại theo từng lần lấy mẫu chặt chẽ hơn nhưng rất tốn kém với SHAP. MLP không được gán trọng số lớp (Spark ML không hỗ trợ).
- **Ngoại tại.** CICIDS2017 (2017) có thể không phản ánh lưu lượng hiện tại; các bộ dữ liệu NIDS chuẩn đều có hạn chế chất lượng đã ghi nhận. Kết luận liên bộ dữ liệu dựa trên *mẫu phân tầng* của CSE-CIC-IDS2018. Chưa đánh giá trước lưu lượng đối kháng. Bản thân việc khử trùng lặp cũng làm thay đổi phân bố benign/attack.
- **Thống kê.** $N=6$ cho *power* thấp: kiểm định hai phía liệt kê đầy đủ chặn p ở mức $2/2^N$, nên $N=6$ chỉ vừa đủ chạm ngưỡng 0,05. Các lần lấy mẫu chồng lấn, vi phạm giả định độc lập của phân phối t — do đó Nadeau–Bengio là căn cứ chính. Tăng N với cross-validation lồng nhau là hướng củng cố tiếp theo.

- Script điều phối phía máy chủ khởi động các vai trò pipeline trên Jetson qua SSH, sinh tải, thu thập log từng nút.
- ≥ 3 lần lặp mỗi cấu hình; lưu lượng khởi động *loại khỏi* cửa sổ đo, để JVM/JIT warmup không làm phồng độ trễ batch đầu.
- Mọi truy vấn chỉ số canh đúng cửa sổ tải, không dùng khoảng thời gian trượt phía sau.
- Phân vị độ trễ tính *theo từng nút* trên mẫu thô từng luồng; p95 mức cụm lấy *thận trọng* theo nút tệ nhất.
- Độ trễ đầu cuối dựa trên đồng hồ đồng bộ NTP và *không* bao gồm chi phí ghi cơ sở dữ liệu.
- Năng lượng: tegrastats, nền idle 30 giây rồi đến cửa sổ tải; chế độ hai nút cộng công suất cả hai board.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

- Script điều phối phía máy chủ khởi động các vai trò pipeline trên Jetson qua SSH, sinh tải, thu thập log từng nút.
- ≥ 3 lần lặp mỗi cấu hình; lưu lượng khởi động *loại khỏi* cửa sổ đo, để JVM/JIT warmup không làm phồng độ trễ batch đầu.
- Mọi truy vấn chỉ số canh đúng cửa sổ tải, không dùng khoảng thời gian trượt phía sau.
- Phân vị độ trễ tính theo từng nút trên mẫu thô từng luồng; p95 mức cụm lấy *thận trọng* theo nút tệ nhất.
- Độ trễ đầu cuối dựa trên đồng hồ đồng bộ NTP và không bao gồm chi phí ghi cơ sở dữ liệu.
- Năng lượng: tegrastats, nền idle 30 giây rồi đến cửa sổ tải; chế độ hai nút cộng công suất cả hai board.

Dự phòng — Vì sao chọn Random Forest cho thiết bị biên?

Cấu hình	F1	Đặc trưng	Dung lượng (MB)	Lý do ở pha biên
Baseline + XGBoost	0.9971	77	2.32	Mắc tham chiều
SHAP Top-30 + XGBoost	0.9970	30	0.003 ¹	Giải thích được
RF Top-30 + XGBoost	0.9922	30	2.52	Xếp hạng nhúng
PCA k=40 + XGBoost	0.9939	40	3.82	Giảm chiều không giám sát

Mô hình triển khai là **SHAP Top-30 với Random Forest** (F1 offline 0.9955, chênh 0.002 so với XGBoost tốt nhất): Random Forest là estimator gốc của Spark ML nên PipelineModel xuất ra nạp được trên board ARM64 mà không cần thư viện native mà tích hợp XGBoost của Spark đòi hỏi. Vector đầu vào có **29** đặc trưng — destination_port nằm trong xếp hạng SHAP nhưng bị loại vì rò rỉ nhãn.

¹Bước lưu mô hình phân tán chỉ ghi metadata cục bộ trên driver nên bảo thiếu dung lượng thật (~2.5 MB).

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Dự phòng — Vì sao chọn Random Forest cho thiết bị biên?

Dự phòng — Vì sao chọn Random Forest cho thiết bị biên?

Cấu hình	F1	Đặc trưng	Dung lượng (MB)	Lý do ở pha biên
Baseline + XGBoost	0.9971	77	2.32	Mắc tham chiều
SHAP Top-30 + XGBoost	0.9970	30	0.003 ¹	Giải thích được
RF Top-30 + XGBoost	0.9922	30	2.52	Xếp hạng nhúng
PCA k=40 + XGBoost	0.9939	40	3.82	Giảm chiều không giám sát

Mô hình triển khai là SHAP Top-30 với Random Forest (F1 offline 0.9955, chênh 0.002 so với XGBoost tốt nhất): Random Forest là estimator gốc của Spark ML nên PipelineModel xuất ra nạp được trên board ARM64 mà không cần thư viện native mà tích hợp XGBoost của Spark đòi hỏi. Vector đầu vào có 29 đặc trưng — destination_port nằm trong xếp hạng SHAP nhưng bị loại vì rò rỉ nhãn.

¹Bước lưu mô hình phân tán chỉ ghi metadata cục bộ trên driver nên bảo thiếu dung lượng thật (~2.5 MB).